COMP3011 Coursework 1
Nutrition and Recipe Analytics API

Student: Joemugen Kirana [201737595]
GitHub Repository: https://github.com/Jyozaa/nutrition-recipe-api
API Documentation PDF: docs/api_documentation.pdf [In public repo]
https://github.com/Jyozaa/nutrition-recipe-api/blob/f8b44e55b70bcd8a49a21749fe7ca4e9a0fcc937/docs/api_documentation.pdf
Presentation Slides: [Insert hosted slides link here]

1. Project Overview

This report presents a database-backed RESTful API for recipe management and nutrition analytics. The system is implemented in Python using FastAPI, SQLAlchemy, and SQLite, and exposes resource-oriented endpoints for recipes, ingredients, categories, and recipe-ingredient links. In addition to CRUD operations, the API computes nutritional summaries, allergen flags, cost estimates, difficulty scores, and filter-driven discovery results. The recipe and nutrition domain was selected because it provides a practical setting in which core web service functionality can be combined with derived analytical behaviour rather than simple record storage alone.

A central design decision was to keep nutritional and dietary facts at ingredient level and derive recipe-level analytics dynamically from the quantities recorded in the recipe-ingredient relationship. In practice, this means that values such as calories, protein, allergens, and estimated cost are not stored redundantly at recipe level, but are calculated from the linked ingredients whenever required. This reduces duplication, improves consistency and makes the data model easier to maintain, since a change to an ingredient's properties automatically affects every recipe that uses it. It also makes the service easier to extend, as additional analytical endpoints can be introduced without restructuring the underlying schema.

The API was designed to support both direct resource management and more user-oriented querying. Alongside standard create, read, update, and delete behaviour, it supports filtering by category, time, diet suitability, allergen-free status, maximum calories, and maximum cost, as well as summary endpoints such as top-protein recipes and lowest-cost recipes. As a result, the project demonstrates not only basic RESTful endpoint construction, but also how a web API can expose useful computed views over relational data.

2. Architecture and Technology Choices

FastAPI was selected because it provides a clean way to build JSON APIs, supports automatic OpenAPI documentation, and integrates well with Pydantic validation. SQLAlchemy was used as the ORM so that the data model could be represented as clear Python classes with relational links, while SQLite was chosen as the database engine because it is lightweight, easy to run locally. The codebase is organised into routers, models, schemas, services, seed scripts, and tests. Routers define endpoint behaviour, models define persistent entities, schemas control request and response validation, and service modules encapsulate analytical logic such as nutrition, allergens, cost, and difficulty calculation. The separation makes it easier to test and explain because validation, storage, and computation are not mixed into a single file.

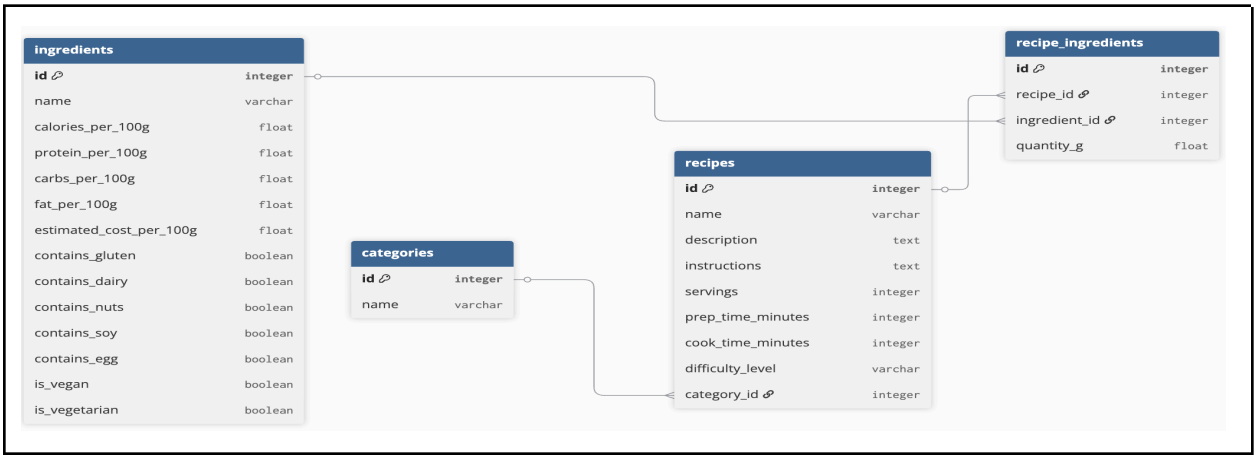

Figure 1. Entity-relationship diagram for the Nutrition and Recipe Analytics API database

3. Data Model and API Design

The relational model contains four entities. Category stores high-level groupings such as breakfast, lunch, dinner, snack, and dessert. Ingredients store reusable facts such as calories per 100g, macronutrients, estimated cost per 100g, allergen flags, and vegan or vegetarian suitability. Recipe stores human-readable recipe information, including instructions, servings, preparation time, cooking time, and a category link. RecipeIngredient is the junction table that resolves the many-to-many relationship between recipes and ingredients by storing the quantity of each ingredient used in grams. The API follows a resource-oriented endpoint structure. Categories, ingredients, and recipes each provide create, read, update, and delete operations through standard HTTP verbs. RecipeIngredient links are managed through nested recipe endpoints because the link has meaning only in the context of a specific recipe. Responses are returned as JSON and the application uses status codes consistently.

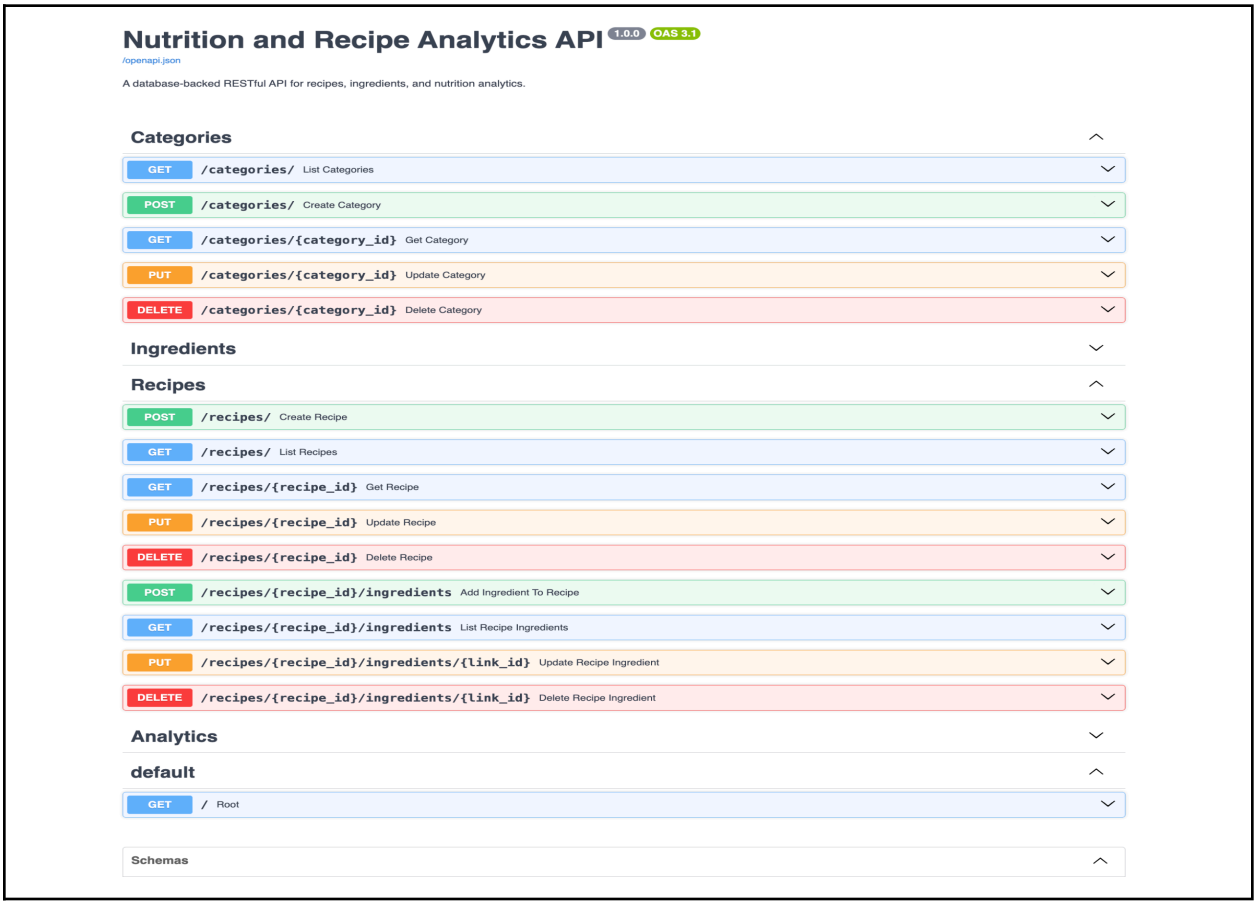

Figure 2. Swagger documentation showing representative endpoint groups and operations.

4. Analytics, Validation, and Testing

The analytical part of the API is implemented as dynamic service-layer computation. Nutritional values are calculated by summing ingredient-level values scaled by quantity in grams and then deriving per-serving values from the recipe serving count. Allergen analysis is derived from ingredient flags, meaning a recipe is marked positive for an allergen if any linked ingredient carries that flag. Cost estimation uses the same per-100g scaling approach as nutrition, while difficulty is estimated through a simple rule-based score using preparation time, cooking time, ingredient count, and instruction complexity. Validation is enforced at two levels. First, Pydantic schemas reject malformed input such as empty names, negative nutrient values, non-positive ingredient quantities, or zero servings. Second, route-level checks enforce domain rules, including preventing duplicate category or ingredient names, blocking duplicate recipe ingredient links, rejecting recipes linked to missing categories, and preventing deletion of categories or ingredients that are still in use. Together, these checks improve reliability and make API behaviour more predictable. Testing was carried out with Pytest and FastAPI TestClient against an isolated in-memory SQLite database. The test suite covers CRUD flows, missing-resource handling, duplicate-link rejection, validation failures, analytics endpoints,

and filter logic. Representative checks include confirming that vegan filters exclude non-vegan recipes, gluten-free filters remove bread-based examples, cost and calorie filters restrict results correctly, and nutrition endpoints return positive computed totals. This approach gave rapid feedback while avoiding interference with the development database.

```
(.venv) Joemugens-MacBook-Pro:nutrition-recipe-api joe$ PYTHONPATH=. pytest tests/test_recipes.py -v
========================================================== test session starts ==========================================================
platform darwin -- Python 3.13.12, pytest-8.4.1, pluggy-1.6.0 -- /Users/joe/Desktop/nutrition-recipe-api/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/joe/Desktop/nutrition-recipe-api
plugins: anyio-4.12.1
collected 8 items

tests/test_recipes.py::test_create_recipe PASSED                                                                                  [ 12%]
tests/test_recipes.py::test_create_recipe_with_invalid_category PASSED                                                            [ 25%]
tests/test_recipes.py::test_add_duplicate_recipe_ingredient_rejected PASSED                                                       [ 37%]
tests/test_recipes.py::test_recipe_detail_contains_ingredients PASSED                                                             [ 50%]
tests/test_recipes.py::test_filter_recipes_by_vegan_only PASSED                                                                   [ 62%]
tests/test_recipes.py::test_filter_recipes_by_gluten_free_only PASSED                                                             [ 75%]
tests/test_recipes.py::test_filter_recipes_by_max_calories PASSED                                                                 [ 87%]
tests/test_recipes.py::test_filter_recipes_by_max_cost PASSED                                                                     [100%]

========================================================== 8 passed in 0.12s ==========================================================
```

```
(.venv) Joemugens-MacBook-Pro:nutrition-recipe-api joe$ PYTHONPATH=. pytest
========================================================== test session starts ==========================================================
platform darwin -- Python 3.13.12, pytest-8.4.1, pluggy-1.6.0
rootdir: /Users/joe/Desktop/nutrition-recipe-api
plugins: anyio-4.12.1
collected 17 items

tests/test_analytics.py ....                                                                                                      [ 23%]
tests/test_categories.py ...                                                                                                      [ 41%]
tests/test_ingredients.py ..                                                                                                      [ 52%]
tests/test_recipes.py ........                                                                                                    [100%]

========================================================== 17 passed in 0.19s ==========================================================
```

Figure 3. Pytest output showing successful execution of the automated test

5. Challenges, Limitations, and Future Development

The main implementation challenge was balancing useful analytical features with a schema that remained simple. The many-to-many relationship between recipes and ingredients was essential, but it also introduced extra validation and testing work because ingredient links had to be updated, deleted, and checked for duplication.Another challenge was deciding whether nutritional values should be stored per recipe or calculated when requested. Dynamic calculation was ultimately preferred because it reduced redundancy and preserved a single source of truth, even though it is less efficient than a precomputed approach at larger scale. The current version also has limitations. It does not include user authentication, user-specific saved recipes, or deployment to a public platform. SQLite was selected for ease of local execution rather than scalability, and the analytical filters are computed in Python rather than pushed into more advanced database queries. The dataset is seeded manually rather than imported from a live public source, which keeps the project manageable but limits the realism and breadth of the data. A natural next stage would be to integrate a public nutrition dataset, add authentication, support pagination and sorting, and improve analytical breadth with richer macro and micro-nutrient coverage. Deployment to a public host and stronger visual dashboards could also improve the end-user experience and make the API easier to demonstrate outside local Development.

6. Generative AI Declaration and Analysis

Generative AI was used throughout planning and refinement in a controlled way. It was used to explore alternative project ideas, structure the API, refine endpoint definitions, suggest validation rules, improve testing strategy, proofread documentation, and help shape this report. AI suggestions were not copied into the submission without review; instead, they were checked against the implemented code and adjusted where necessary. The final architecture, code choices, endpoint selection, and written explanations reflect my own understanding of the system.

The main tool used was ChatGPT. It was used in four broad ways. First, it supported early project ideation and scope selection. At the planning stage, AI was used to explore several possible API ideas and compare them in terms of feasibility, technical depth, and alignment with the brief. This was useful because the coursework is open-ended, and AI helped narrow the problem space efficiently. AI was also used for design refinement and structured problem solving. Examples include discussing data models, endpoint organisation, testing strategy, filtering logic, and how to structure the project into routers, schemas, models, and services. Its main benefit was helping to surface alternatives quickly and making the reasoning process more explicit. For example, AI suggestions helped compare whether nutrition values should be stored directly at recipe level or derived dynamically from ingredient-level data.

AI was used for debugging and implementation support. This included troubleshooting issues such as the SQLite test database setup, validating route behaviour, improving filtering logic, and strengthening validation. In these cases, AI accelerated debugging by suggesting likely causes and possible fixes. However, these suggestions were not trusted blindly. They were checked against actual error messages, tested in the local environment, and only kept if they worked correctly in the context of the codebase. It was also utilized for documentation and report support, including improving wording, clarifying explanations, and helping structure the technical report and README. This use was productive because it helped improve clarity and organisation, especially when summarising design choices and testing strategy. At the same time, this introduced the risk of producing generic explanations that sounded polished but were not closely tied to the actual implementation. To reduce that risk, the written sections were cross-checked against the implemented code, endpoints, and tests so that the final report remained an accurate account of the implemented system.

The main benefits of using AI in this project were speed, breadth of idea generation, and support during debugging and structuring. It reduced time spent getting unstuck, helped compare alternatives more quickly, and supported more systematic planning than might otherwise have been possible. In particular, it was valuable in moving beyond a basic CRUD design into a more feature-complete API with analytics, filtering, and better testing coverage.

7. Appendix and Submission Links

Repository: https://github.com/Jyozaa/nutrition-recipe-api
API documentation PDF:https://github.com/Jyozaa/nutrition-recipe-api/blob/f8b44e55b70bcd8a49a21749fe7ca4e9a0fcc937/docs/api_documentation.pdf
Presentation slides: [Insert hosted slide URL]

Appendix A. Generative AI Usage Log

Tool	Purpose	Example outcome
ChatGPT (OpenAI)	project ideation	narrowed project to recipe/nutrition analytics API
ChatGPT (OpenAI)	debugging	helped fix test database/import issues
ChatGPT (OpenAI)	documentation drafting	drafted API docs and report wording

Relevant GPT chat logs

(a) **Me**: I was tasked to design, implement, and present a fully functional data-driven web API with database integration. And the minimum of implementing at least one data model involving all four elements of CRUD (Create, Read, Update and Delete) functionality linked to a database. Suggest some API ideas for this project

GPT:
Yes — here are the strongest options I'd recommend for you.

1. Study Planner / Productivity Analytics API
This is the one I'd recommend most for you.
Why it is strong
It is easy to understand, easy to demo, and very realistic to finish. It also naturally supports CRUD plus analytics
Core models
User, Module, Task / Assessment, StudySession, HabitLog

CRUD resources
create, read, update, delete tasks, create, read, update, delete study sessions, create, read, update, delete modules
Interesting analytics endpoints
streaks per habit, total study hours by week, busiest module, overdue tasks, completion rate by module, productivity heatmap data by day/hour

2. Recipe / Nutrition Analytics API
This is probably the safest "impressive but still doable" idea.
Core models
Recipe, Ingredient, Category, NutritionInfo, UserSavedRecipe

CRUD resources, CRUD recipes, CRUD ingredients, CRUD categories

Interesting analytics endpoints
calories per recipe, macro breakdown, allergen warnings, cheapest recipes, highest protein recipes, filter by ingredient, cuisine, calories, diet type

—----------

(b) **Me**: I am leaning towards Option 2

GPT:
Great choice. Option 2: Nutrition and Recipe Analytics API is one of the best fits for the brief because the brief explicitly names it as an example and says it should support ingredient/recipe

CRUD plus computed values like calories, macronutrients, allergen warnings, and cost or difficulty estimates.
I'd shape it into something like this:
Project idea
Working title
NutriTrack API
A RESTful API for managing recipes and ingredients, backed by a SQL database, with analytics endpoints that calculate nutrition, allergens, cost, and recipe difficulty.

What your project should do
Your API should let a user:

- create recipes
- read recipes
- update recipes
- delete recipes
- manage ingredients
- attach ingredients to recipes
- query calculated nutrition values
- get allergen flags
- estimate cost and difficulty
- filter/search recipes

So this is not just a plain CRUD app. It is a CRUD + analytics API, which is much better for marks.

Recommended scope
Keep it focused and do one main resource very well.
Main CRUD resource
Recipe
Supporting resources

- Ingredient, RecipeIngredient (join table between recipe and ingredient), Category

Recommended tech stack, for this coursework, I'd recommend:
Best stack choice
Python, FastAPI, SQLAlchemy, SQLite for development, optionally PostgreSQL later if you want, Pydantic for validation, Swagger / OpenAPI docs automatically from FastAPI, pytest for testing
Why this is a strong choice
FastAPI is very good for:
clean REST endpoints, JSON responses, automatic API docs, input validation, good status-code handling
---

(c) **Me**: Walk me through how to start this project. Be as detailed as possible.
Overall build plan

GPT:
We'll do this in this order:

1. create the project folder
2. create a Python virtual environment
3. install packages
4. create the folder structure
5. add the starter code file by file
6. run the API locally
7. test it in Swagger docs
8. add CRUD for recipes, ingredients, categories
9. add recipe-ingredient linking
10. add analytics endpoints
11. add seed data
12. add tests
13. clean the README

Your folder structure should look like this

```
nutrition-recipe-api/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── database.py
│   ├── models/
│   ├── schemas/
│   ├── routers/
│   ├── services/
│   └── seed/
├── tests/
├── requirements.txt
├── README.md
└── .gitignore
```

GPT then proceeds to provide starter code for:

- app/database.py
- app/models/category.py
- app/models/ingredient.py
- app/models/recipe.py
- app/models/recipe_ingredient.py
- app/models/__init__.py
- app/schemas/category.py
- app/schemas/ingredient.py
- app/schemas/recipe.py
- app/schemas/analytics.py
- app/services/nutrition_service.py

- app/services/allergen_service.py
- app/services/cost_service.py
- app/services/difficulty_service.py
- app/routers/categories.py
- app/routers/ingredients.py
- app/routers/recipes.py
- app/routers/analytics.py
- app/main.py
- tests/test_categories.py
- tests/test_recipes.py
- tests/test_analytics.py
- tests/conftest.py

Next steps after starter code
Once the app above is running, your next coding tasks should be:
1. Improve recipe detail output
At the moment recipe responses are simple. Later, include:

- category name
- ingredient list
- maybe analytics summary

2. Add duplicate-link protection
Do not allow the same ingredient to be added twice to the same recipe unless updating quantity.
3. Add richer filtering
For example:

- /recipes?category_id=1
- /recipes?max_prep_time=20
- /recipes?is_vegan=true
- /recipes?contains_nuts=false

4. Add analytics summary endpoints
Examples:

- top highest protein recipes
- cheapest recipes
- vegan recipes
- recipe count by category

These make the project look more analytical, which fits the example idea in the brief very well.
5. Add proper tests
You should eventually test:
successful creation, invalid input, 404 not found, delete behaviour, analytics correctness

6. Improve docs and screenshots

(d) **Me**: Attached is an error I obtained after executing the command “pytest”

```
=================================================== short test summary info ===================================================
ERROR tests/test_analytics.py::test_recipe_cost_endpoint - sqlalchemy.exc.OperationalError: (sqlite3.OperationalError) a
ttempt to write a readonly database
ERROR tests/test_analytics.py::test_recipe_allergens_endpoint - sqlalchemy.exc.OperationalError: (sqlite3.OperationalErr
or) attempt to write a readonly database
ERROR tests/test_analytics.py::test_top_protein_endpoint - sqlalchemy.exc.OperationalError: (sqlite3.OperationalError) a
ttempt to write a readonly database
ERROR tests/test_categories.py::test_create_category - sqlalchemy.exc.OperationalError: (sqlite3.OperationalError) attem
pt to write a readonly database
ERROR tests/test_categories.py::test_duplicate_category_rejected - sqlalchemy.exc.OperationalError: (sqlite3.Operational
Error) attempt to write a readonly database
ERROR tests/test_categories.py::test_get_missing_category_returns_404 - sqlalchemy.exc.OperationalError: (sqlite3.Operat
ionalError) attempt to write a readonly database
ERROR tests/test_ingredients.py::test_create_ingredient - sqlalchemy.exc.OperationalError: (sqlite3.OperationalError) at
tempt to write a readonly database
ERROR tests/test_ingredients.py::test_negative_nutrition_rejected - sqlalchemy.exc.OperationalError: (sqlite3.Operationa
lError) attempt to write a readonly database
ERROR tests/test_recipes.py::test_create_recipe - sqlalchemy.exc.OperationalError: (sqlite3.OperationalError) attempt to
 write a readonly database
ERROR tests/test_recipes.py::test_create_recipe_with_invalid_category - sqlalchemy.exc.OperationalError: (sqlite3.Operat
ionalError) attempt to write a readonly database
ERROR tests/test_recipes.py::test_add_duplicate_recipe_ingredient_rejected - sqlalchemy.exc.OperationalError: (sqlite3.O
perationalError) attempt to write a readonly database
ERROR tests/test_recipes.py::test_recipe_detail_contains_ingredients - sqlalchemy.exc.OperationalError: (sqlite3.Operati
onalError) attempt to write a readonly database
========================================== 1 passed, 12 errors in 1.64s ==========================================
```

GPT:

That error is coming from the **test database setup**, not from your API logic.

`attempt to write a readonly database` usually happens here because the SQLite file-based test DB in `tests/conftest.py` is being handled badly across tests. The most reliable fix is to **stop using a file for the test DB** and switch to an **in-memory SQLite test database**.

That is the cleanest fix.

What to change

Go to your project root:

```bash
cd ~/Desktop/nutrition-recipe-api
source .venv/bin/activate
```

Open:

```bash
nano tests/conftest.py
```

Replace the whole file with this:

```python
import pytest
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.pool import StaticPool
from sqlalchemy.orm import sessionmaker

from app.database import Base, get_db
from app.main import app

TEST_DATABASE_URL = "sqlite://"

engine = create_engine(
    TEST_DATABASE_URL,
    connect_args={"check_same_thread": False},
    poolclass=StaticPool,
)

TestingSessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)


def override_get_db():
    db = TestingSessionLocal()
    try:
        yield db
    finally:
        db.close()


@pytest.fixture(scope="function")
def client():
    Base.metadata.drop_all(bind=engine)
    Base.metadata.create_all(bind=engine)

    app.dependency_overrides[get_db] = override_get_db

    with TestClient(app) as test_client:
        yield test_client

    app.dependency_overrides.clear()
```

Then run “pytest” again

—----------

(e) **Me**: Help me understand the report structure and the best way to write it

GPT:

Your report needs to do six core jobs:

1. explain your design and architecture choices
2. justify your technology stack
3. explain your testing approach
4. reflect on challenges and lessons learned
5. discuss limitations and future improvements
6. include a GenAI declaration and analysis of usage

On top of that, the report must be:

- logically structured with labelled sections
- written in clear English and professional communication
- include diagrams where appropriate
- include at least attempts at referencing libraries, tutorials, or datasets used.

So the best way to write it is not as “what I did day by day,” but as:
problem → design choices → implementation decisions → evaluation → reflection

Best report structure
This is the structure I would recommend for your project.
1. Introduction / Project Overview
Purpose: tell the examiner what the API is and what it does.
Keep this short. In one paragraph, explain:

- what the API is for
- what entities it manages
- what makes it more than basic CRUD
- what deliverables exist alongside it, like GitHub/API docs/slides

For your project, this section should briefly mention:

- recipes
- ingredients
- categories
- recipe-ingredient links
- analytics such as nutrition, allergens, cost, difficulty, filtering

This section answers: what did you build?

2. System Design and Architecture
Purpose: explain how the project is organised.
This is one of the most important sections because the brief explicitly asks for design and architectural choices.
Here you explain:

- the overall structure of the app

- separation into models, schemas, routers, services, seed, and tests
- why you separated business logic into services
- how requests flow through the system

A clean way to explain it:

- models define database structure
- schemas validate request/response bodies
- routers expose HTTP endpoints
- services hold calculation logic for nutrition, allergens, cost, difficulty
- seed provides demo data
- tests verify behaviour

This section answers: how is the project organised internally?
A simple diagram would help here:

- client → router → schema validation → service logic → database

That would satisfy the “include diagrams where appropriate” part nicely.

3. Technology Stack and Justification
Purpose: explain why you chose FastAPI, SQLAlchemy, SQLite, etc.
The brief explicitly wants your personal choices for the technology stack and justification of programming language, frameworks, and database.
This section should not just list tools. It should explain trade-offs.
For example:

- Python for fast development and readability
- FastAPI for clean REST endpoint development, validation, and automatic OpenAPI docs
- SQLAlchemy for ORM-based relational modelling
- SQLite because it is lightweight and easy to run locally
- Pytest for automated testing

Make this sound like your reasoning, not a generic product description.
This section answers: why did you build it this way?

4. API and Data Model Design
Purpose: explain the core entities and how they relate. This section should cover:

- Category
- Ingredient
- Recipe
- RecipeIngredient

Then explain why the junction table matters:

- recipes contain many ingredients
- ingredients can belong to many recipes
- quantities are stored in the link table

Then explain your design choice of deriving nutrition/cost/allergen info from ingredient data instead of storing duplicate totals directly in recipes.
That is a strong design justification because it shows:

- reduced redundancy
- improved consistency
- easier maintenance

This section answers: what data did you model, and why that model?
A small ER-style diagram would work very well here.

5. Key Functionality / Implementation Highlights
Purpose: show what is important about the implementation without dumping code.
Do not describe every endpoint one by one unless the report becomes too thin otherwise.
Instead, group things by function:
- CRUD endpoints
- analytics endpoints
- filtering
- validation and error handling
- seeding support

Use this section to highlight the more interesting parts of your API:
- nutrition totals calculated from ingredient quantities
- allergen detection based on ingredient flags
- cost estimation
- filtering by dietary and numeric conditions
- proper status codes and JSON responses

This section answers: what are the most important technical features?

6. Testing Approach
Purpose: explain how you verified the API.
The brief explicitly asks for your testing approach.
This section should cover:
- use of pytest
- what kinds of tests you wrote
- why those tests matter

A good structure:
- CRUD tests
- validation tests
- missing-resource tests
- duplicate-handling tests
- analytics correctness tests
- filtering tests

Do not just say “I used pytest.” Explain what risk each group of tests helps reduce.
This section answers: how do you know the API works?

7. Challenges and Lessons Learned
Purpose: reflect honestly.

The brief explicitly wants challenges and lessons learned.
This section is where you show maturity.
Good challenges for your project might include:

- modelling many-to-many recipe/ingredient relationships
- avoiding redundant nutrition data
- keeping service logic separate from routers
- designing useful filter combinations
- writing tests for calculated values
- handling environment/setup issues, such as Python version dependency issues

Lessons learned might include:

- why separation of concerns matters
- why testing calculated endpoints is different from testing CRUD
- how schema validation reduces downstream bugs
- how derived values improve consistency but increase implementation complexity

This section answers: what was difficult, and what did you learn from it?

8. Limitations and Future Improvements
Purpose: show critical evaluation.
The brief explicitly requires limitations and potential areas for improvement or future development.
This section is not about apologising for your work. It is about showing awareness.
Examples for your project:

- no authentication
- limited pagination/search
- SQLite is fine for local/demo use but not ideal for larger production scenarios
- allergen modelling is simplified
- cost estimation is only approximate
- difficulty scoring may be heuristic-based rather than data-driven
- analytics could be expanded further

Future improvements:

- user accounts
- saved recipes
- image uploads
- advanced search
- deployment
- caching
- CI pipeline
- richer nutritional coverage
- public dataset integration

This section answers: what would you improve next, and why?

9. Generative AI Declaration and Analysis
Purpose: satisfy the brief and do it properly.

The brief requires a Generative AI declaration and thoughtful analysis of its usage, and the Minerva submission must include conversation logs as an appendix.
This section should say:

- which tools you used
- what you used them for
- how you validated outputs
- what parts remained your own work
- what benefits and limitations AI had in this project

Do not make this one sentence only. The brief wants analysis, not just mention.
This section answers: how did you use AI responsibly and meaningfully?
—----------